大小端：同一段内存，两台机器读出不同的数

嵌入式 Linux 手记　2026-11-09　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。同一段内存里存着 `int x = 0x12345678`，用 `char *` 读它的第一个字节：

```c
#include <stdio.h>

int main(void) {
    int x = 0x12345678;
    unsigned char *p = (unsigned char *)&x;
    printf("第一个字节 = 0x%02X\n", p[0]);
    return 0;
}
```

x86 与多数 ARM 单片机上打印 `0x78`，换成一台大端机器打印 `0x12`。变量的值没变（`printf("%x", x)` 两边都是 12345678），变的是这 4 个字节在内存里的摆放顺序。这就是字节序（endianness）：同一份数据，两台机器读出不同的数。这一篇讲清它的定义、记忆法、四个咬人现场，以及一套端序无关的写法。

一、定义与记忆法

一个多字节数据（比如 4 字节的 `int`）在内存里必然按地址顺序逐字节存放，争议只在：最高位字节放哪头。大端（Big-endian）：高字节放低地址，按地址递增读出来就是人类书写的顺序（`12 34 56 78`）；小端（Little-endian）：低字节放低地址，字节顺序恰好倒过来（`78 56 34 12`）。x86 是小端；ARM 体系多数芯片按小端跑，Cortex-M 则固定小端；网络字节序规定为大端。

记忆法一句话：大端符合人的书写习惯，小端符合机器的取用习惯。小端的好处是取最低字节不用知道总宽度（`char*` 读第一个字节永远是"数值低位"），类型收窄时地址不用动。两种顺序没有对错，历史各选一边，痛苦留给了跨机器的工程师。

顺带一个帮助记忆的典故："大端""小端"出自《格列佛游记》小人国之争——磕鸡蛋该从大头敲还是小头敲，两派打了两场战争。网络先驱 Cohen 用它命名两种字节序，讽刺工程师为字节顺序站队的执念；名字流传了下来，战争也确实延续到了今天（只是改为在代码评审里互喷）。

漏转 `htons` 的病可以精确演示：端口 5000 的十六进制是 0x1388，内存里小端摆放为 `88 13`；直接把这 4 个十六进制位当大端读，就成了 0x8813——端口"变成了"34835。数字语法完全

合法，连接永远失败，这类 bug 只能靠对协议的字节级 dump 才能定位。



图：0x12345678 的两种摆放与判本机端序三法。指针法直观、联合体法严谨、编译期宏零开销。

二、四个咬人现场

现场一，跨机通信。网络协议一律大端（RFC 明文的"网络字节序"），x86 主机是小端，16/32 位整数跨网前必须过转换四件套：发送 `htons` / `htonl`，接收 `ntohs` / `ntohl`。漏转的典型症状是"数字对但高低位颠倒"：端口 5000（0x1388）发出去变成 0x8813，连不上还查不出语法错。

现场二，文件格式。PNG 大端、BMP 小端、ZIP 小端。解析文件头时若不按格式声明的端序读，魔数都对不上。跨平台存档、协议帧、遥测日志同理：文件与协议里每个多字节字段都必须显式约定端序。

现场三，联合体与指针读字节。判断本机端序的经典代码正是利用"按字节重读"：指针强转法（开场代码）最直观，但按篇 13 引过的严格别名规则，用联合体成员访问更严谨（C 明文允许通过联合体成员互看对象表示）。工程上更推荐编译期内建宏：GCC/Clang 预定义 `__BYTE_ORDER__`，编译期就确定端序、选好代码路径，零运行时开销。

现场四，DMA 与外设寄存器。网络控制器的描述符、USB 包的字段，端序由硬件设计冻结：CPU 侧可配的端序若与外设约定不一致，读到的寄存器值全部错位。选型与驱动移植时，字节序是数据手册"必读章节"，不是脚注。

三、端序无关的写法

最稳的写法根本不关心本机端序：按字节移位组装。

```
/* 从大端字节流 buf 取 32 位：两端机器结果一致 */
uint32_t v = ((uint32_t)buf[0] << 24)
           | ((uint32_t)buf[1] << 16)
           | ((uint32_t)buf[2] << 8)
           |  (uint32_t)buf[3];
/* 序列化同理：v >> 24、(v >> 16) & 0xFF ... 逐字节写入 */
```

移位组装在大小端机器上结果完全一致，代价是几条指令，换来的是"这份代码不需要知道自己在哪台机器上编译"。嵌入式串口协议（传感器、Modbus 帧）常用这招，从根上规避端序问题。反例是直接把结构体指针压在字节流上强转，端序、对齐、填充三个雷一起埋，篇 24 刚讲过其中两个。



图：四个咬人现场与端序无关写法。现场各不相同，解法一致：按字节约定，显式转换。

四、两个补丁口径

口径一，htons 四件套只管 16/32 位整数：64 位有系统扩展（Windows 有 `htonll`，Linux/BSD 有 `be64toh` / `htobe64`），浮点数没有标准转换函数：IEEE 754 的字节序跟随主机，跨机传浮点同样走序列化，别指望一个函数打天下。口径二，`char` 单字节数据与端序无关：被咬的永远是多字节类型（`int`、`long`、指针、浮点），协议设计时把字段按字节流切好，端序问题就只剩整数转换一处。工程检查清单里因此多一条：评审跨机传输代码时，凡看到裸指针强转读多字节字段，先问端序，再看对齐。

写在最后

1. 字节序只关乎多字节数据的摆放：大端高字节在低地址（网络序、符合书写），小端低字节在低地址（x86、Cortex-M 固定小端）。
2. 四个咬人现场：网络通信（漏 htons 端口变乱码）、文件格式（PNG 大端/BMP 小端）、联合体与指针读字节（判端序三法：指针、联合体、__BYTE_ORDER__ 宏）、DMA 与外设寄存器。
3. 端序无关写法是移位组装：逐字节移位拼出 32 位，两端机器结果一致；结构体指针直接压字节流是端序+对齐+填充三雷同埋。
4. htons 四件套只管 16/32 位整数，浮点无标准转换，跨机传浮点走序列化；单字节 char 与端序无关。
5. 判端序留给编译期（__BYTE_ORDER__）最优，零运行时开销；运行期判别用联合体比指针强转更严谨（不碰别名红线）。

内存系列的最后一块拼图与性能有关：同样的二维数组，行遍历比列遍历快几倍到几十倍。下一篇《换个循环顺序快 10 倍：CPU 缓存与局部性》，看 64 字节缓存行、预取器与局部性如何改写循环的写法，系列 2 的收官之战。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。